

ENTERPRISE SECURITY ASSESSMENT LAB

CLOUDGOAT OBJECTIVE AND EC2 CLEANUP VERIFICATION

Professional methodology report for post-lab resource destruction and EC2 verification

Module	Cloud Security Assessment
Document Type	Individual Methodology Report
Phase	Phase 7
Evidence Source	13-cloudgoat-ec2-cleanup-verified.png
Publication Type	GitHub Portfolio - Redacted Lab Version
Prepared For	04-cloud-security/README.md documentation set
File Name	07-cloudgoat-objective-and-ec2-cleanup-verification-methodology-report.pdf

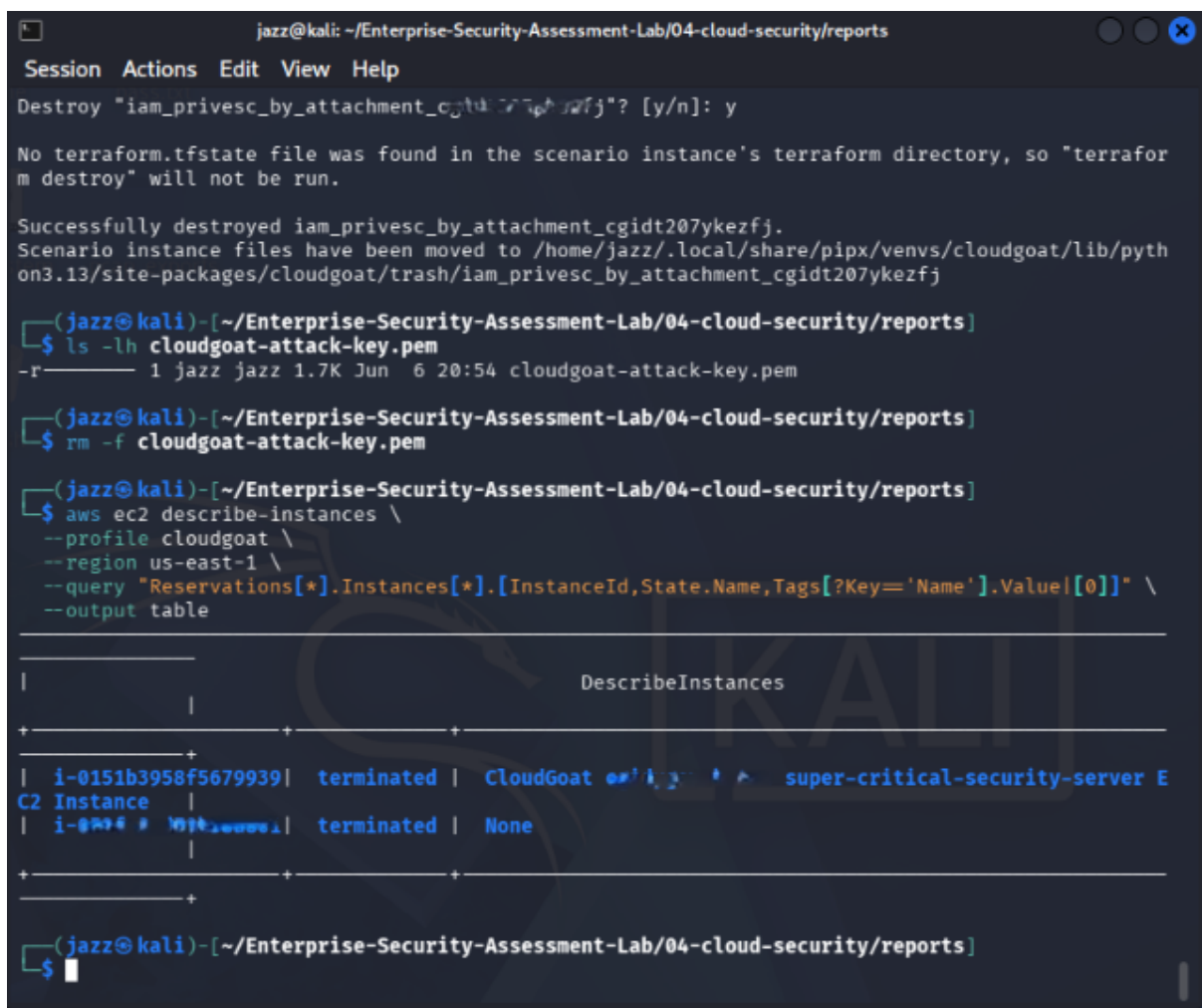
1. Executive Purpose

Cloud security labs can create real billable resources. This methodology section documents the operational discipline of destroying the vulnerable lab scenario and verifying that EC2 resources are no longer active.

2. Scope and Evidence Mapping

Testing objective	Complete the CloudGoat lab objective and verify that EC2 resources were cleaned up afterwards.
Cloud platform	Amazon Web Services (AWS)
Lab context	CloudGoat / authorized vulnerable cloud lab environment
Primary tools	CloudGoat, AWS CLI, EC2 API
Evidence file(s)	13-cloudgoat-ec2-cleanup-verified.png
Result type	EC2 resource cleanup verification completed

3. Embedded Evidence Screenshot(s)



```
jazz@kali: ~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports
Session Actions Edit View Help
Destroy "iam_privsec_by_attachment_cgidd207ykezfj"? [y/n]: y

No terraform.tfstate file was found in the scenario instance's terraform directory, so "terraform destroy" will not be run.

Successfully destroyed iam_privsec_by_attachment_cgidd207ykezfj.
Scenario instance files have been moved to /home/jazz/.local/share/pipx/venvs/cloudgoat/lib/python3.13/site-packages/cloudgoat/trash/iam_privsec_by_attachment_cgidd207ykezfj

(jazz@kali)-[~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports]
$ ls -lh cloudgoat-attack-key.pem
-r----- 1 jazz jazz 1.7K Jun  6 20:54 cloudgoat-attack-key.pem

(jazz@kali)-[~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports]
$ rm -f cloudgoat-attack-key.pem

(jazz@kali)-[~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports]
$ aws ec2 describe-instances \
  --profile cloudgoat \
  --region us-east-1 \
  --query "Reservations[*].Instances[*].[InstanceId,State.Name,Tags[?Key='Name'].Value[0]]" \
  --output table

DescribeInstances
+-----+-----+-----+
| i-0151b3958f5679939 | terminated | CloudGoat super-critical-security-server E C2 Instance |
| i-0496f0101b0000001 | terminated | None |
+-----+-----+-----+
```

Figure 1 - EC2 cleanup verification after CloudGoat testing.

4. Methodology and Execution Workflow

After completing the CloudGoat scenario, the tester runs the scenario destroy command, then independently verifies EC2 state through AWS CLI. This prevents accidental resource sprawl and reduces both billing and security exposure.

Command / workflow used

```
cloudgoat destroy <SCENARIO_NAME>
aws ec2 describe-instances --profile $PROFILE --region $REGION
```

Execution notes

- Destroy the exact scenario that was deployed.
- Verify EC2 state after destroy rather than assuming cleanup completed.
- Check IAM, S3, and CloudFormation resources as well if the scenario created them.
- Commit evidence only after sensitive identifiers are removed.

5. Sanitized Tool Output Style

The following output example is sanitized. It documents the expected evidence style without exposing AWS account IDs, ARN values, AccessKeyId values, SecretAccessKey values, session tokens, private bucket names, or billing details.

```
CloudGoat destroy completed.
EC2 describe-instances returned no unexpected active lab instances.
Cleanup verification captured for GitHub evidence.
```

6. Professional Interpretation

The screenshot supports responsible cloud testing by proving that cleanup verification was performed after the vulnerable lab activity.

7. Risk Analysis

Risk Area	Explanation
Billing risk	Unterminated EC2 instances can generate unexpected costs.
Exposure risk	Vulnerable lab resources left running may remain reachable.
Credential risk	Unused lab infrastructure may retain roles, keys, or public endpoints.

8. Defensive Recommendations

- Destroy vulnerable lab scenarios immediately after testing.
- Verify EC2, IAM, S3, and CloudFormation cleanup.
- Use budget alerts and cost anomaly detection.
- Use tags to identify lab resources.
- Avoid leaving public IPs and permissive security groups active.

9. GitHub Redaction and Publication Guidance

Before publishing this document or the underlying screenshot evidence, verify that the following values are removed, blurred, or replaced with placeholders. This is especially important because cloud evidence can contain values that

are valid beyond the lab session if not rotated or destroyed.

Sensitive value	Required handling
AWS Account ID	Blur or replace with <REDACTED_ACCOUNT_ID>.
ARN values	Redact account number and private role/user names where needed.
AccessKeyId / SecretAccessKey	Must never be visible. Rotate immediately if exposed.
SessionToken	Must be fully redacted because temporary credentials can still be abused during validity period.
Bucket names / resource names	Redact if they identify private infrastructure or are still active.
Instance IDs / public IPs	Redact if active or tied to a personal AWS account.

10. Evidence Quality Checklist

- The screenshot is readable and aligned within the page margins.
- The evidence filename matches the GitHub link exactly.
- The command/output supports the stated result type.
- No AWS credentials, session tokens, account numbers, or private resource identifiers are visible.
- The report explains both the offensive testing value and the defensive remediation value.